easybits

Data Table Extraction

Setup guide

Prepared by easybits

August 2026

> This guide walks through the exact pipeline setup used to reach 100% extraction accuracy on a multi-row tax assessment table. The single most important decision is the **shape of your response structure**. Get that right and the row-bleed problem disappears before you touch a single field description.

The core idea: one object per row, not parallel lists

The instinctive way to model a table is one list per column: a list of names, a list of emails, a list of addresses. The problem is that nothing links position 4 in the name list to position 4 in the email list. The moment the engine skips a single empty cell in one column, that column shifts by one and every row beneath it is wrong. This is the classic “a value jumped in from another row” symptom.

The fix is structural. You create a single field called `records`, mark it as an **array**, and set its type to **object**. Each column then lives as a **nested field** inside that object, so one array entry holds all the values for one physical row. Because the fields live together in the same object, they cannot drift apart.

Recommended structure

```
// records is an ARRAY of OBJECTS — one object per table row
{
  "records": [
    {
      "si":                   "1",
      "new_tin":              "622589468028",
      "asses_name":           "Dr. Elijah Taylor",
      "father_name":          "Mr. Lucas Miller",
      "contact_email_address": "NULL",
      "current_address":      "Building #3, Flat #D-2, Maplewood Residential Area, North District
    }
  ]
}
```

Field	Type in the builder	Why
`records`	`object [ ]`	The one array you create: an array of objects. Every field below is nested inside it, and this container is what keeps each row's values together.
↳ `new_tin`	`string`	Nested field. Must be a string to preserve leading zeros (e.g. `048304172125`); a number type would drop them.
↳ `asses_name`, `father_name`	`string`	Nested fields. Free text, including prefixes such as Dr., Md., Late.
↳ `contact_email_address`	`string`	Nested field. String so the literal word `NULL` can be returned as text for empty cells.
↳ `current_address`	`string`	Nested field. One string even when the address wraps across two visual lines.
↳ `si`	`number` (recommended)	Nested field. The row's sequence number; a number type matches the column as shown in the builder. Use string only if you need to keep formatting.

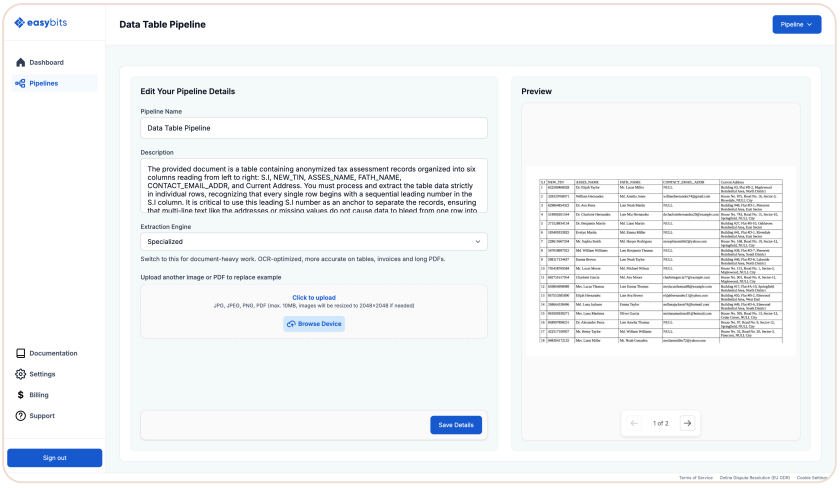
Do not use string arrays per field. A shape like `{ new_tin: [...], asses_name: [...] }` keeps every column in its own separate list and relies on all the lists staying the same length, which is exactly what makes rows desync. The only array you want is `records` itself: one array of objects, with each column as a nested field inside it.

1 Create the pipeline and upload an example

In **Pipelines** → **Create a Pipeline**, give the pipeline a name and upload an example of the document you will be extracting from – it appears in the preview on the right. Fill the **Description** field with a short, plain description of that document. The detailed extraction rules do not go here; they belong on the `records` field in the next step.

`Pipeline description` DESCRIBES THE UPLOADED EXAMPLE DOCUMENT

A tax assessment register presented as a multi-column table. Each row is one assessed individual and lists their taxpayer identification number, name, father's name, contact email, and current address.



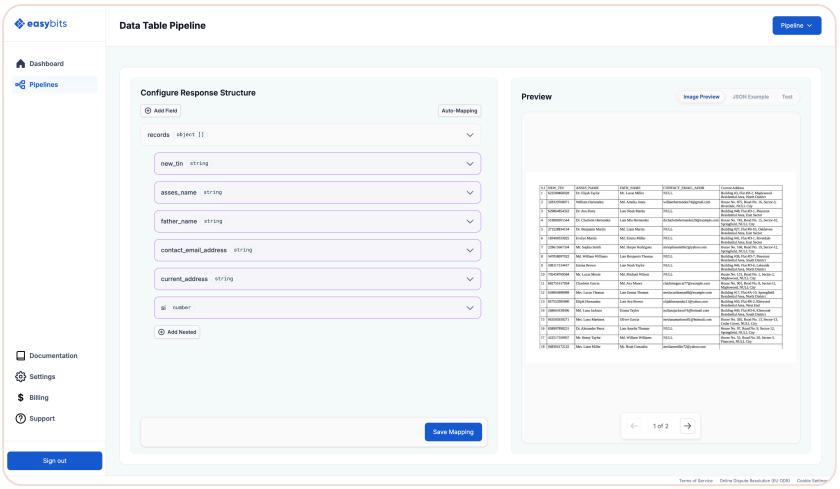
Pipeline setup: the uploaded example document appears in the preview, the **Description** field describes it, and the **Extraction Engine** dropdown is set to Specialized.

2 Build the response structure

This is the step after pipeline setup, where you map the fields. Add a single field named `records`, mark it as an **array** and set its type to **object** (so it shows as `object [ ]`). Give `records` its own description – this is where the detailed extraction rules live – then use **Add Nested** to add each column as a nested field inside it.

`records` OBJECT [ ] – FIELD DESCRIPTION

This document is a table of anonymized tax assessment records with six columns, left to right: S.I, NEW_TIN, ASSES_NAME, FATH_NAME, CONTACT_EMAIL_ADDR, and Current Address. Extract one object per row. Each row begins with a sequential number in the S.I column. Some cells contain the literal text NULL, which should be read as the text NULL rather than as an empty value. Some addresses wrap across two visual lines but belong to a single cell, and the words "NULL City" inside an address are part of a real place name, not an empty value.



`records` set to `object [ ]` with each column added underneath as a nested field. Note `si` comes in as a `number`; the rest are `string`.

3 Write each field description

Each nested field gets its own short description telling the engine which column to read and how to handle its edge cases. Below is each one as used, with a note on what it defends against.

`si` STRING

The sequential row number from the leftmost S.I column (1, 2, 3, and so on). Extract as a string. Every other field in this object must come from the same row as this S.I number.

Guards against: rows merging or splitting. The S.I is the anchor every other field references.

`new_tin` STRING

The Taxpayer Identification Number from the NEW_TIN column, from the same row as this object's S.I number. Extract exactly as printed, as a string, preserving all leading zeros. Never convert it to a number or drop leading zeros. It is a 12-digit value.

Guards against: leading-zero loss on values like `063585939271` and `048304172125`.

`asses_name` STRING

The assessed individual's full name from the ASSES_NAME column, including any prefix such as Dr., Mr., Mrs., or Md., from the same row as this object's S.I number.

Guards against: dropped honorifics. "Md." is part of the name here, not a typo.

`father_name` STRING

The father's name from the FATH_NAME column, including any prefix such as Md., Mr., or Late, from the same row as this object's S.I number.

Guards against: stripping the "Late" prefix, which carries meaning and must be kept.

`contact_email_address` STRING

The email address from the CONTACT_EMAIL_ADDR column for this row. If the cell contains the literal text NULL, return exactly NULL as text; do not leave it empty and do not invent an address. From the same row as this object's S.I number.

Guards against: hallucinated emails and inconsistent handling of empty cells.

`current_address` **STRING**

The full address from the Current Address column for this row, combining building or house number, flat number, road, sector, area, and district into one string even when it wraps across two lines. Keep the text exactly as printed, including any occurrence of the words NULL City, which is part of the address and must not be removed. From the same row as this object's S.I number.

Guards against: two-line addresses being split, and the place name "NULL City" being wrongly treated as an empty value.

The two faces of "NULL". In the email column, `NULL` means the cell is empty and should be returned as the text NULL. Inside an address, `NULL City` is a real place name and must be kept verbatim. Spell this distinction out in the descriptions, as above, or the engine will guess.

4 Choosing the extraction engine

The **Extraction Engine** dropdown selects which model runs behind the pipeline. Both use the same schema and descriptions; they differ in what they are tuned for.

General Gemini

A strong all-round choice for mixed content: forms, short documents, screenshots, and everyday extraction where layout is simple.

- Fast, broadly capable
- Good default for most pipelines

Specialized Mistral

OCR-optimised and more accurate on document-heavy work: dense tables, invoices, and long or multi-page PDFs.

- Best for tables and scanned pages
- The right pick for this pipeline

For a data table like this one, **Specialized** is the recommended engine. In testing it read a 22-row table, including a row split across a page break, at full accuracy.

Testing tips

- Build a small reference of the correct values and compare the extraction against it, cell by cell, rather than eyeballing the output.
- Run the same document two or three times. Extraction is not fully deterministic, so confirm your accuracy holds across runs before trusting it.
- Watch page breaks. When a table spans pages, the column header can repeat mid-record; a good description tells the engine to ignore repeated header text inside a cell.
- Test both engines on the same document so you can compare accuracy and speed for your specific document type.